

|                     |                                     |
|---------------------|-------------------------------------|
| Course Name         | : Digital Systems Design Laboratory |
| Name                | : <b>ARYAN SINGH SIKARWAR</b>       |
| Register no         | : <b>24BCE0403</b>                  |
| Slot/Batch timing   | : <b>L41+42</b>                     |
| Lab faculty Name    | : <b>DILIP KUMAR CHOUDHARY</b>      |
| Theory Faculty Name | : <b>DILIP KUMAR CHOUDHARY</b>      |

## Aim

To write, simulate, and verify Verilog HDL code for the following:

- Basic logic gates using gate-level and dataflow modeling.
- The Boolean expression

$F(A,B,C)=A+BC+ABC$  using gate-level and dataflow modeling.

- Half adder and half subtractor circuits using a test bench in Modelsim.

## Tool Required

- Modelsim Software
- Verilog HDL (Hardware Description Language)

## Theory

A Hardware Description Language (HDL) is used to describe and model digital systems. Verilog is one such HDL.

- **Modeling Styles:**

- **Gate Level Modeling:** This style describes a circuit by specifying the logic gates (e.g., and, not, or) and how they are interconnected.
- **Dataflow Modeling:** This style focuses on the flow of data between registers and how it is processed. It uses continuous assignments to express logic, often with the

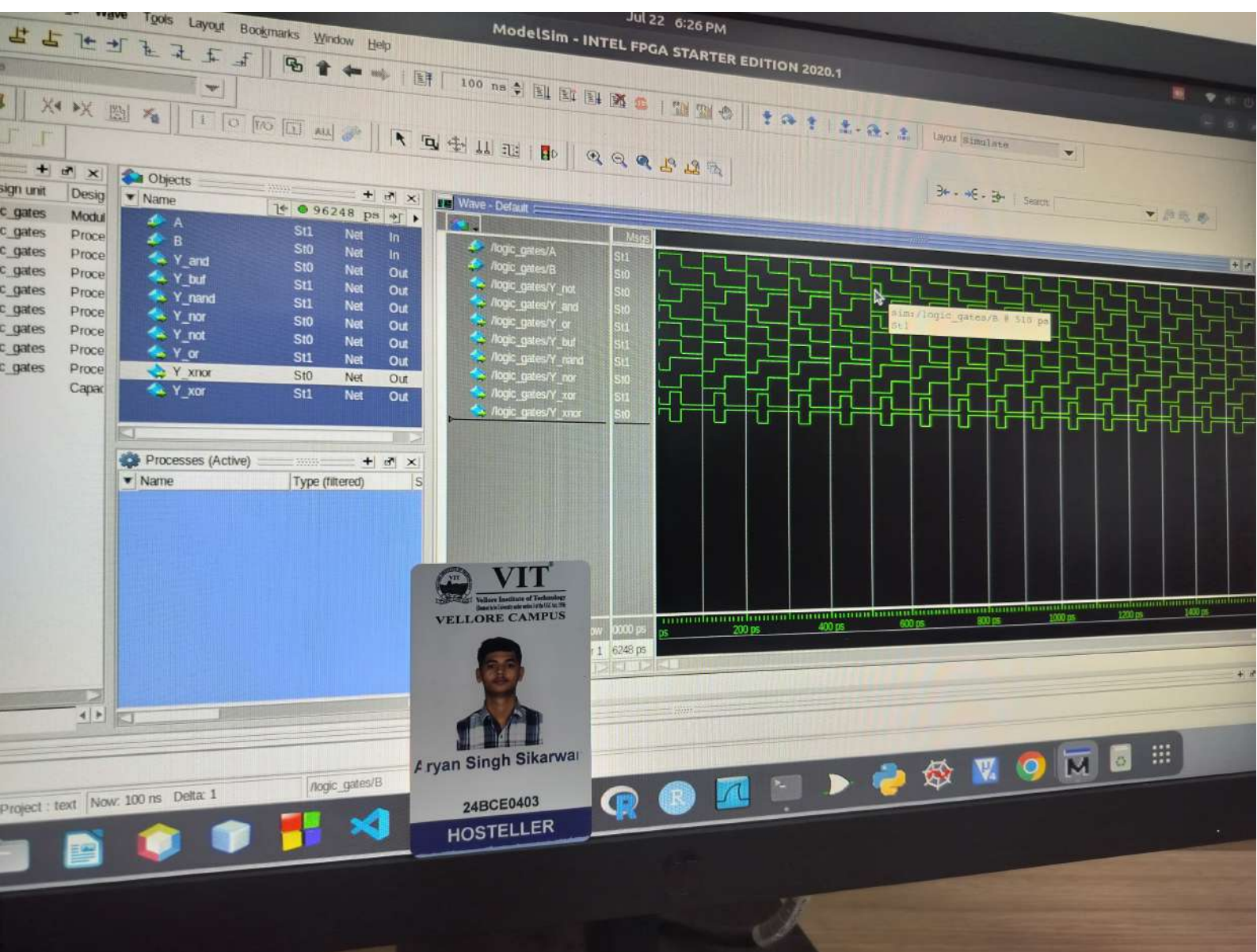
assign keyword.

- **Logic Gates:** These are fundamental electronic circuits that operate on one or more input signals to produce an output signal. Basic gates include AND, OR, and NOT, while NAND and NOR are known as universal gates because any digital circuit can be constructed from them.
- **Half Adder:** This is a combinational circuit designed to add two single-bit numbers. It produces two outputs: a Sum (

$A \oplus B$ ) and a Carry ( $AB$ ).

- **Half Subtractor:** This is a digital logic circuit that performs the subtraction of two single-bit binary numbers. It has two outputs: a Difference (

$A \oplus B$ ) and a Borrow ( $A \cdot B$ ).





## (Que-1) (All logic gates)

CLASSMATE  
Date \_\_\_\_\_  
Page \_\_\_\_\_

(\*) Problem Statement :-

⇒ write verilog code in one file for all gates using gate level modeling and then verify its operation.

Ans

Code :-

```
Module All_gate (Y_and, Y_or, Y_not,  
Y_and, Y_nand, Y_nor, A, B);  
input A, B;
```

```
output Y_and, Y_or, Y_not, Y_nand,  
Y_nor;
```

```
and (Y_and, A, B);
```

```
or (Y_or, A, B);
```

```
not (Y_not, A);
```

```
nand (Y_nand, A, B);
```

```
nor (Y_nor, A, B);
```

```
end module
```

(\*) Data flow :-

```
module all_gate (output Y_and, Y_or, Y_nand,  
Y_nor, input A, B);
```

```
assign Y_and = A & B;
```

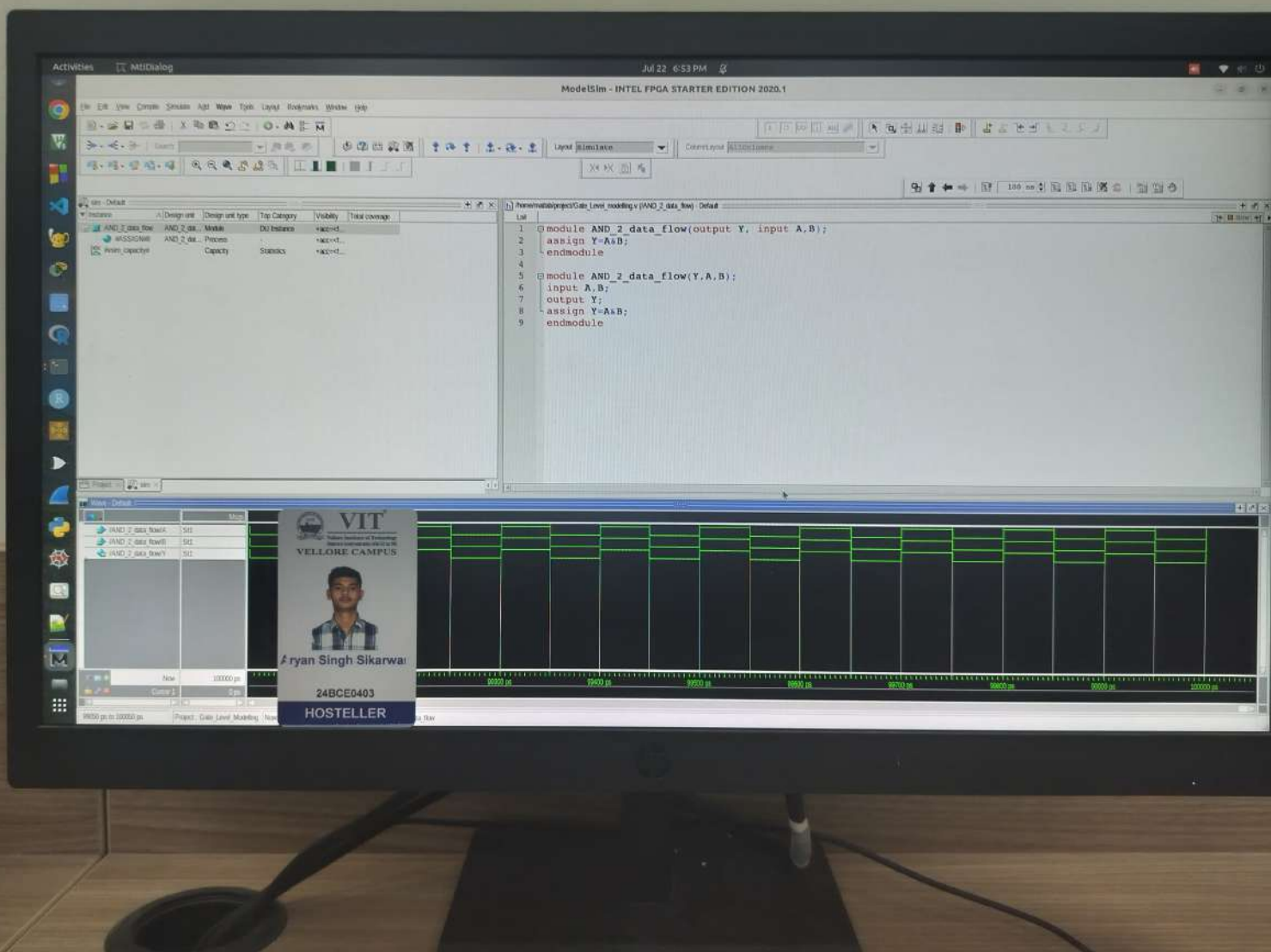
```
assign Y_or = A | B;
```

```
assign Y_not = !A;
```

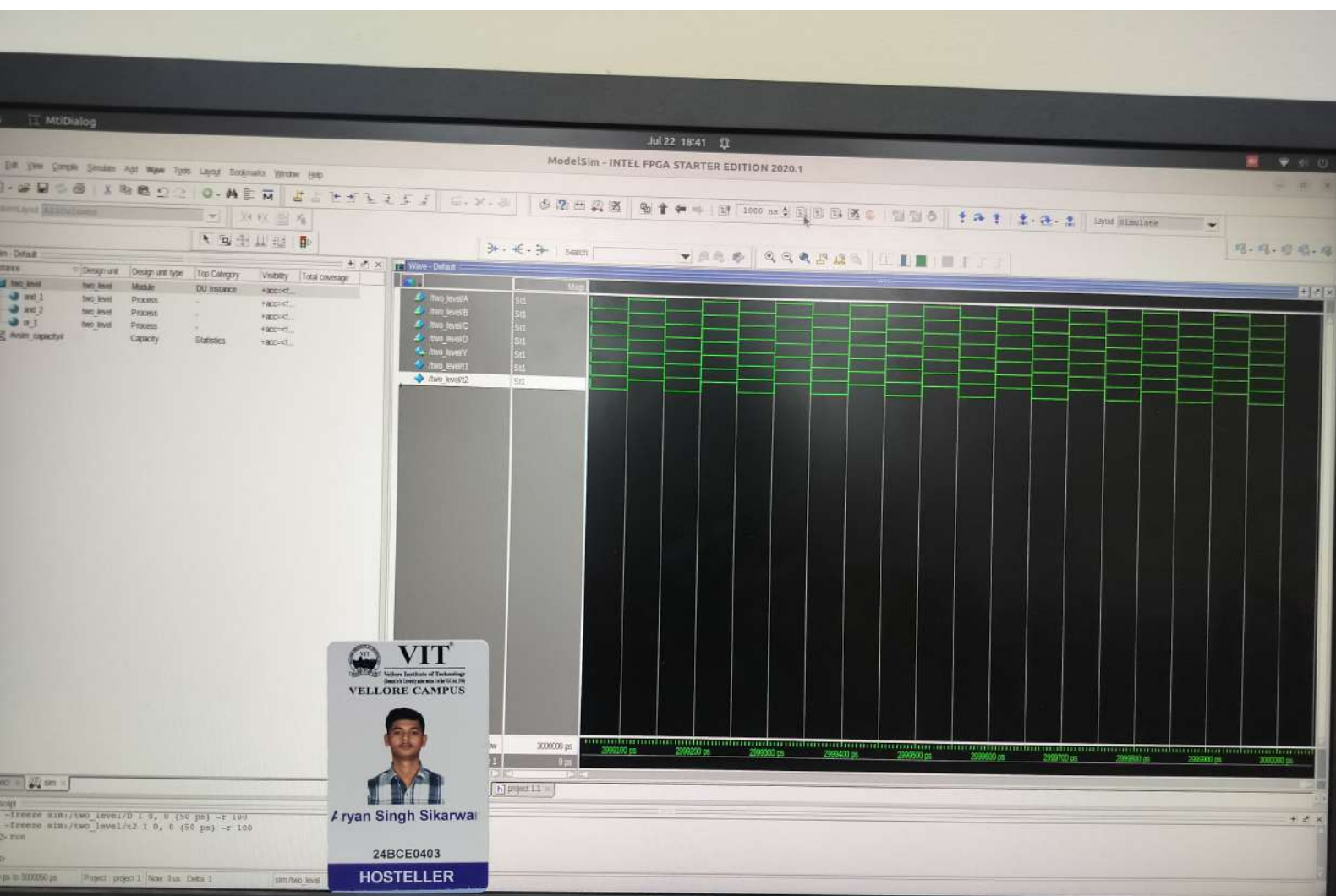
```
assign Y_nand = !(A & B);
```

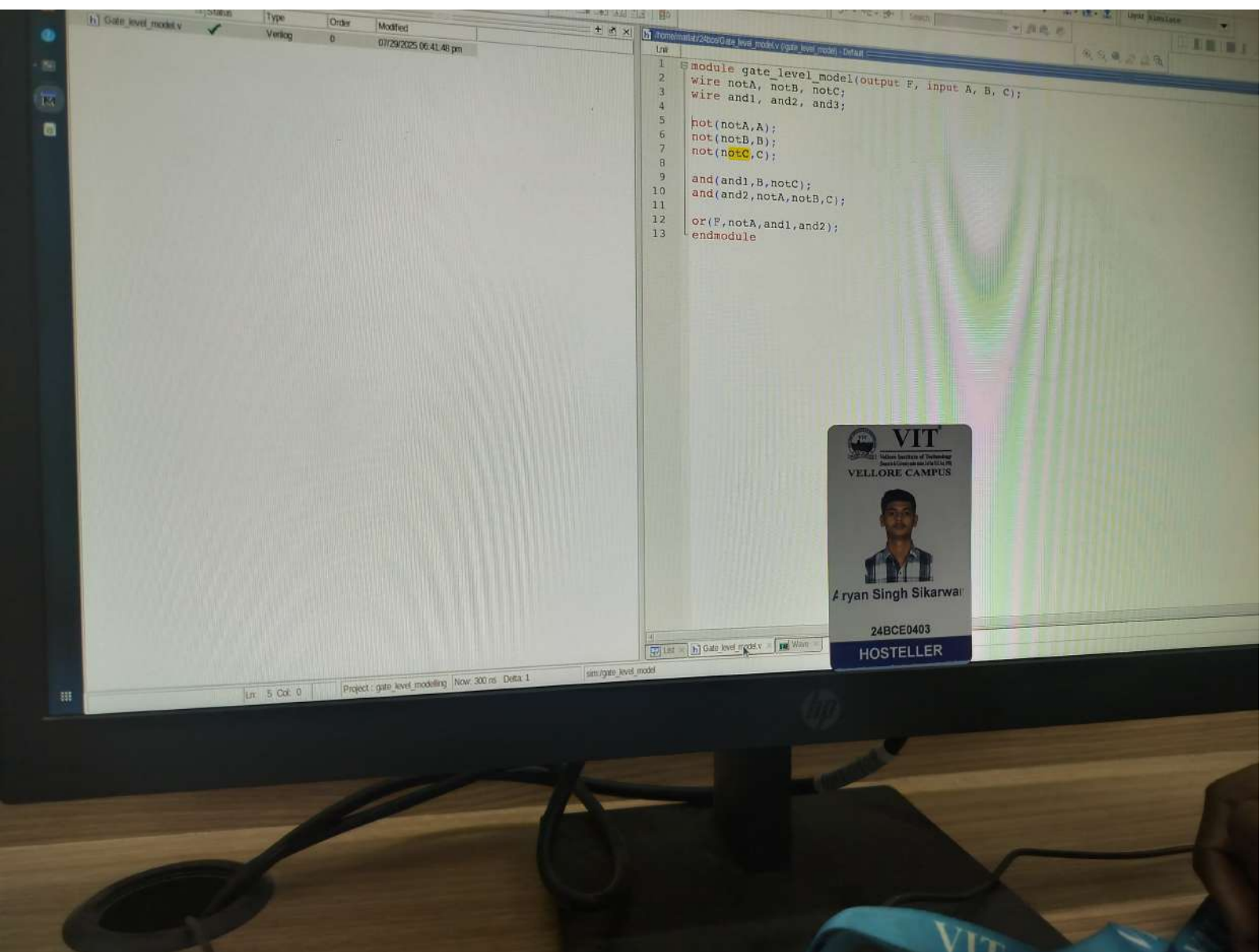
```
assign Y_nor = !(A | B);
```

```
endmodule;
```

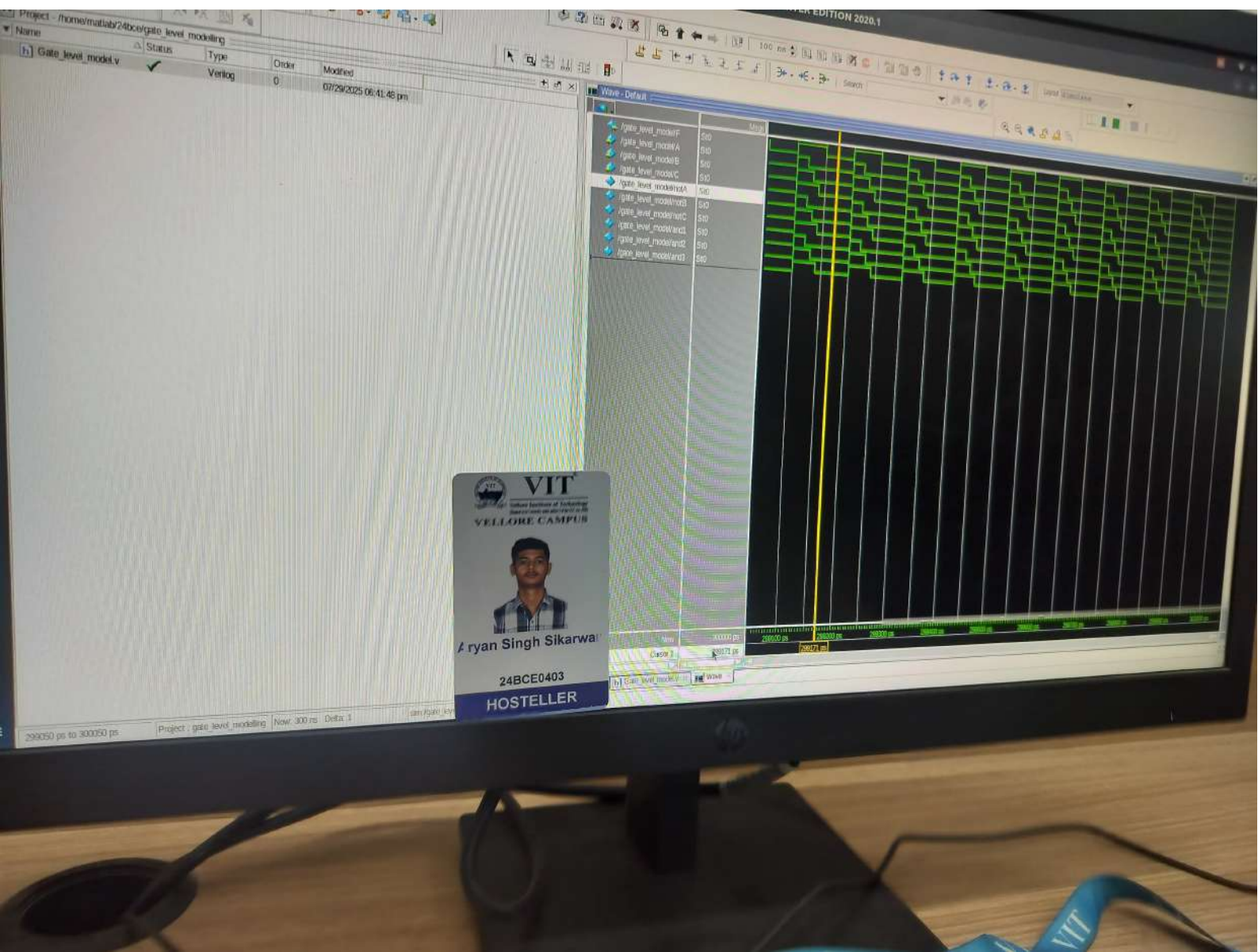












\* problem 2 :-

=> write Verilog code in one file for all gates using Data flow Modeling and then verify its operation. two level modeling.

```
code => module two-level (Y, A, B, C, D);  
    Input A, B, C, D;  
    Output Y;  
    time t1, t2;  
  
    and and-1 (t1, A, B);  
    and and-2 (t2, A, D);  
    or or-1 (t1, t2);  
end module
```

\* Data flow Modeling :- Q1

```
=> module AND-2-data-flow  
    (output Y, output A, B);  
    assign Y = A & B;  
end module
```

*o/p verified  
Dip  
29-07-2023*

\* Data flow modeling :- Q2

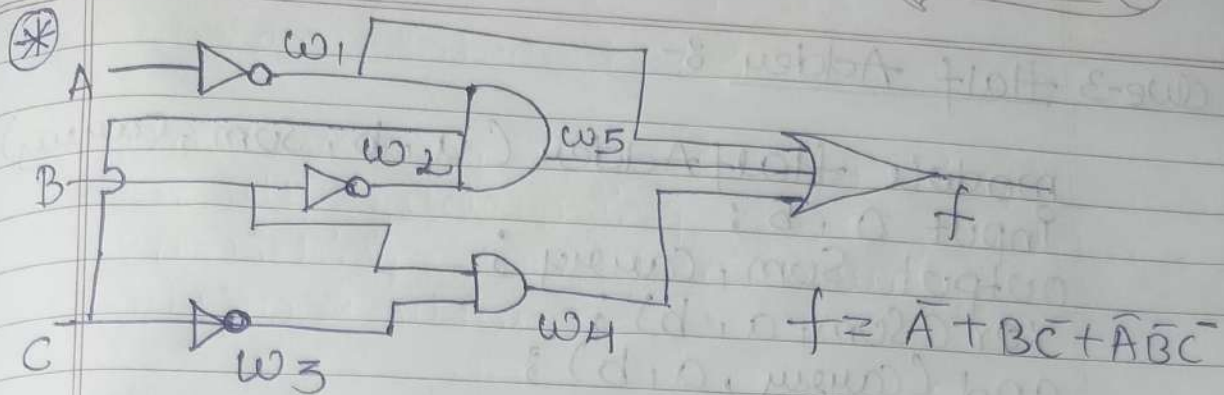
```
module data_flow_F (output F, input A, B,  
    assign F = !A | (B & !C) | (A & B & C);  
end module
```



(Que-2)

classmate

Date  
Page



⇒ module ex ( f, A, B, C );  
Input A, B, C;  
output f;

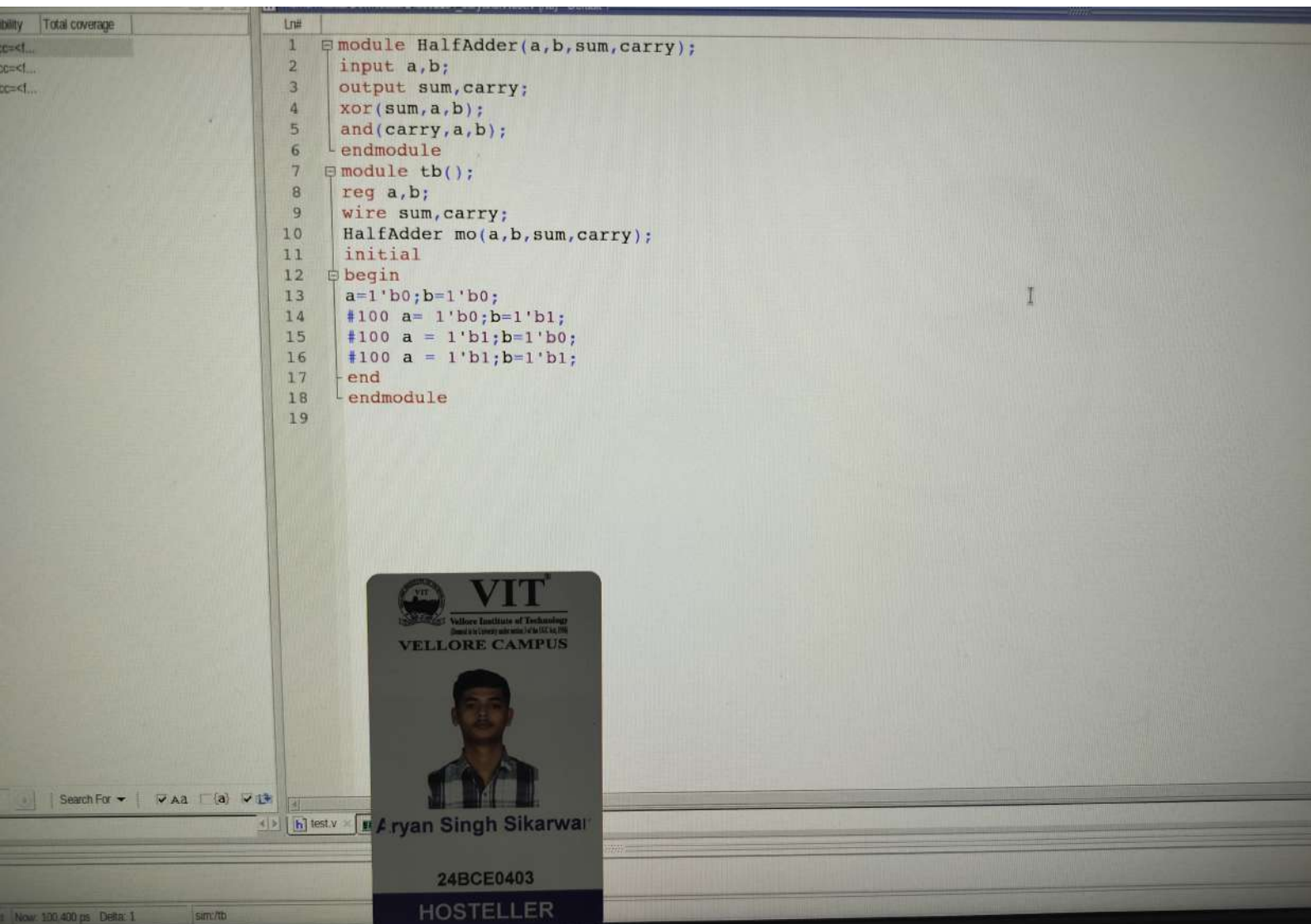
Wire w1, w2, w3, w4, w5;  
not m0 (w1, A);  
not m1 (w2, B);  
and m2 (w3, C);

and m3 (w4, B, w3);  
and m4 (w5, w1, C, w4);  
or m5 (f, w5, w4, w4);  
endmodule

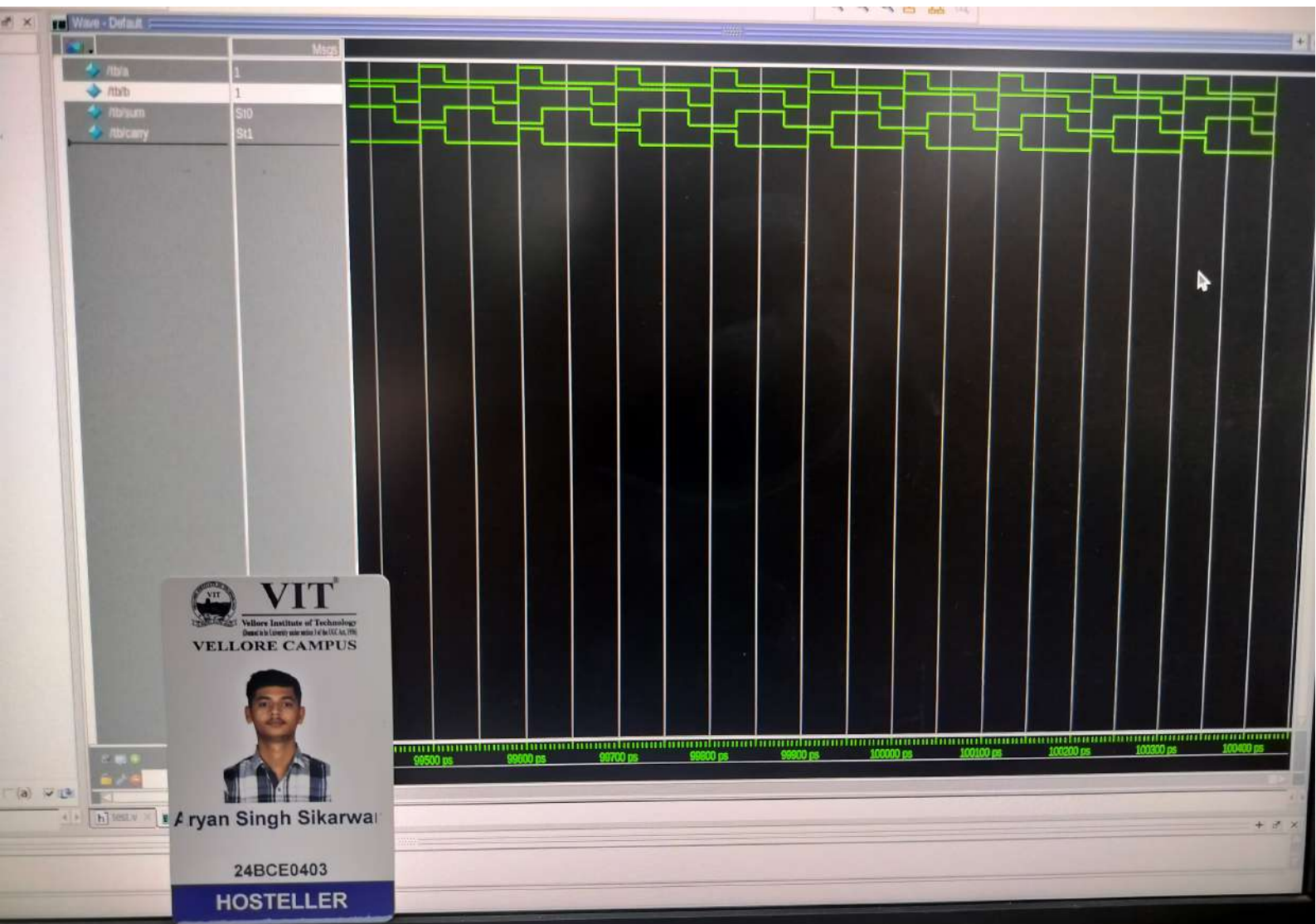
| A | B | C | $\bar{A}$ | $\bar{B}$ | $\bar{C}$ | $B\bar{C}$ | $\bar{A}\bar{B}\bar{C}$ | $f = \bar{A} + \bar{B}\bar{C} + \bar{A}\bar{B}\bar{C}$ |
|---|---|---|-----------|-----------|-----------|------------|-------------------------|--------------------------------------------------------|
| 0 | 0 | 0 | 1         | 1         | 1         | 0          | 0                       | 1                                                      |
| 0 | 0 | 1 | 1         | 1         | 0         | 0          | 1                       | 1                                                      |
| 0 | 1 | 0 | 1         | 0         | 1         | 1          | 0                       | 1                                                      |
| 0 | 1 | 1 | 1         | 0         | 0         | 0          | 0                       | 1                                                      |
| 1 | 0 | 0 | 0         | 1         | 1         | 0          | 0                       | 0                                                      |
| 1 | 0 | 1 | 0         | 1         | 0         | 0          | 0                       | 0                                                      |
| 1 | 1 | 0 | 0         | 0         | 1         | 1          | 0                       | 1                                                      |
| 1 | 1 | 1 | 0         | 0         | 0         | 0          | 0                       | 0                                                      |

P.N: 24 BCE 0403  
OIP verified

29-07-2024







### Que-3 Half Adder :-

```
module HalfAdder (a, b, Sum, Carry);  
    Input a, b;  
    output Sum, Carry;  
    xor (Sum, a, b);  
    and (Carry, a, b);  
endmodule  
  
module tb();  
    reg a, b;  
    wire Sum, Carry;  
    HalfAdder ma(a, b, Sum, Carry);  
    initial  
    begin  
        a = 1'b0; b = 1'b0;  
        #100 a = 1'b0; b = 1'b1;  
        #100 a = 1'b1; b = 1'b0;  
        #100 a = 1'b1; b = 1'b1;  
    end  
endmodule
```

### 2) Truth Table :-

| A | B | Sum(s) | Carry (c) |
|---|---|--------|-----------|
| 0 | 0 | 0      | 0         |
| 0 | 1 | 1      | 0         |
| 1 | 0 | 1      | 0         |
| 1 | 1 | 0      | 1         |

R.N: 24 BCE0902  
O/P Verified

Dilip  
05-08-2025

```

Ln#
1  module Halfsubtractor(a,b,difference,borrow);
2  input a,b;
3  output difference,borrow;
4  xor(difference,a,b);
5  assign borrow=(-a&b);
6  endmodule
7  module tb_s();
8  reg a,b;
9  wire difference,borrow;
10 Halfsubtractor mo(a,b,difference,borrow);
11 initial
12 begin
13 a=1'b0;b=1'b0;
14 #100 a= 1'b0;b=1'b1;
15 #100 a = 1'b1;b=1'b0;
16 #100 a = 1'b1;b=1'b1;
17 end
18 endmodule
19
20

```

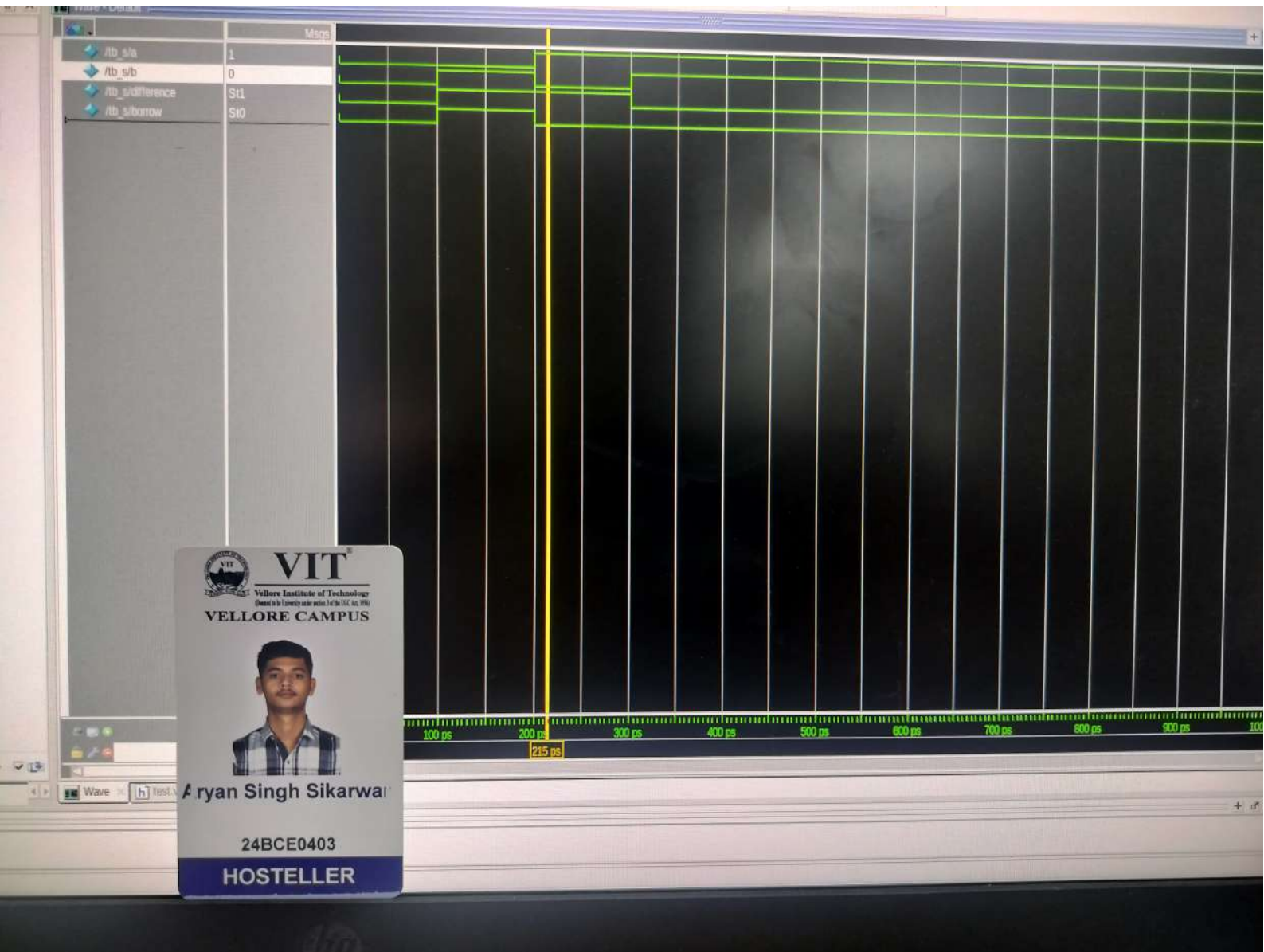

**VIT**  
 Vellore Institute of Technology  
(Approved by the Government under section 3 of the UGC Act, 1956)  
**VELLORE CAMPUS**



Aryan Singh Sikarwal

24BCE0403





Que-2, part (B) :- Half Subtractor :-

1) Gate level :-

```
module half_subtractor (diff, borrow, A, B);  
    input A, B;  
    output diff, borrow;  
    wire not_A;  
    xor (diff, A, B);  
    not (not_A, A);  
    and (borrow, not_A, B);  
endmodule
```

2) Data flow :-

```
module half_subtractor (output diff, borrow,  
    input A, B);  
    assign diff = A ^ B;  
    assign borrow = !A & B;  
endmodule
```

3) Truth table :-

| A | B | Difference | Borrow |
|---|---|------------|--------|
| 0 | 0 | 0          | 0      |
| 0 | 1 | 1          | 1      |
| 1 | 0 | 1          | 0      |
| 1 | 1 | 0          | 0      |